```
function [retmap,retvisited,retsteps] = bfs(mapfile,startlocation,targetlocation)

% Convert the map file into a MATLAB matrix
map = map_convert(mapfile);
retmap = map;

% Get dimensions of the map
[maprows,mapcols] = size(map);

% Create a matrix to keep track of visited cells
visited = false(maprows,mapcols);

% Keep track of the parent of every visited cell
parent = zeros(maprows,mapcols,2);

% BFS queue
queue = startlocation;

% Mark the starting location as visited
visited(startlocation(1),startlocation(2)) = true;

% Store the order in which cells were visited
retvisited = [];

% Directions:
%       up
%       down
%       left
%       right
directions = [
    -1  0
     1  0
     0 -1
     0  1
];

% BFS
while ~isempty(queue)

    % Take the FIRST item from the queue
    current = queue(1,:);
    queue(1,:) = [];

    % Record that this cell was visited
    retvisited = [retvisited; current];

    % Have we reached the target?
    if current(1) == targetlocation(1) && ...
       current(2) == targetlocation(2)

        break;
    end

    % Check each neighbouring cell
    for k = 1:4

        neighbour = current + directions(k,:);

        row = neighbour(1);
        col = neighbour(2);

```

```
        % Check that neighbour is inside the map
        if row >= 1 && row <= maprows && ...
           col >= 1 && col <= mapcols

            % Check that it is not a wall
            if map(row,col) == 0

                % Check that it has not already been visited
                if ~visited(row,col)

                    % Mark as visited
                    visited(row,col) = true;

                    % Remember where we came from
                    parent(row,col,1) = current(1);
                    parent(row,col,2) = current(2);

                    % Add neighbour to the END of the queue
                    queue = [queue; neighbour];

                end
            end
        end
    end
end

% Reconstruct the shortest path
retsteps = [];

% Check whether target was reached
if visited(targetlocation(1),targetlocation(2))

    current = targetlocation;

    while ~(current(1) == startlocation(1) && ...
            current(2) == startlocation(2))

        % Add current cell to the path
        retsteps = [current; retsteps];

        % Move to the parent
        parentrow = parent(current(1),current(2),1);
        parentcol = parent(current(1),current(2),2);

        current = [parentrow parentcol];

    end

    % Add starting location
    retsteps = [startlocation; retsteps];

end

plotmap(retmap, retsteps);

end


function placestep(position,i)

% This function will plot a yellow rectangle
```

```
% and print a number in the rectangle.

position = [16-position(1) position(2)];

position = [position(2)+0.1 position(1)+0.1];

rectangle('Position',[position,0.8,0.8],'FaceColor','y');

c = sprintf('%d',i);

text(position(1)+0.2,position(2)+0.2,c,'FontSize',10);

end
```